LangChain Docs

Comience > Flujos de trabajo y agentes

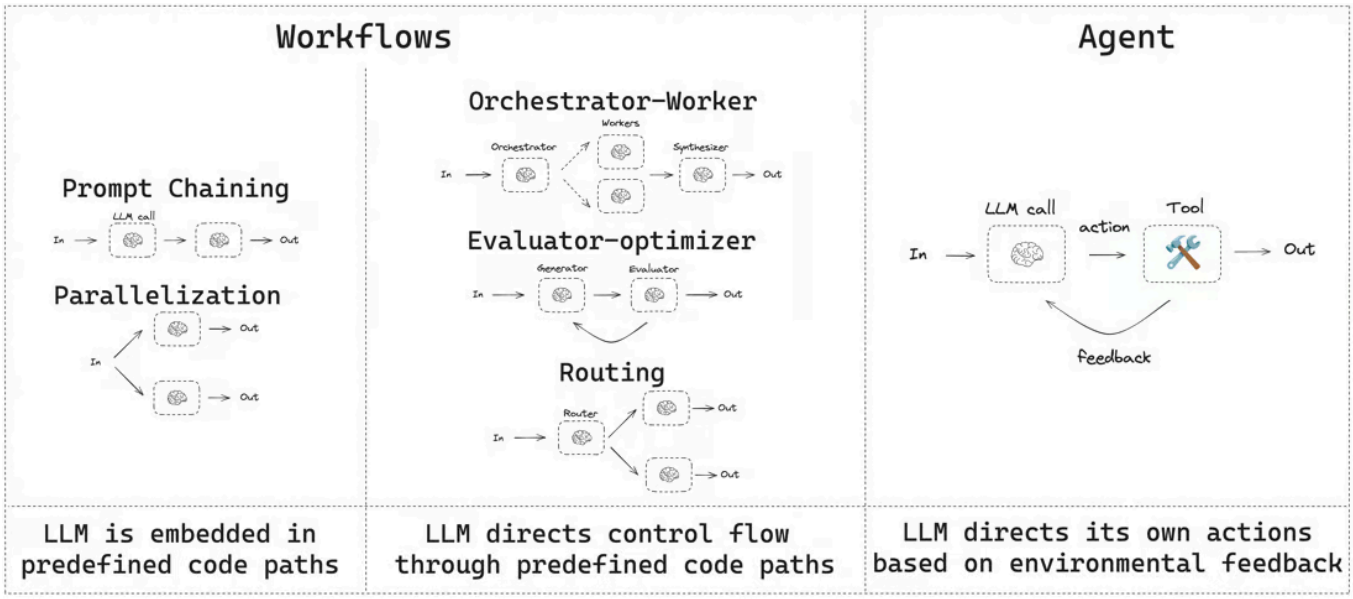
Comience

Flujos de trabajo y agentes

Copiar página

Esta guía repasa los flujos de trabajo y los patrones de agentes más comunes.

- Los flujos de trabajo tienen rutas de código predeterminadas y están diseñados para operar en un orden determinado.
- Los agentes son dinámicos y definen sus propios procesos y el uso de sus herramientas.



LangGraph ofrece varias ventajas a la hora de crear agentes y flujos de trabajo, entre las que se incluyen la persistencia , la transmisión de datos y la compatibilidad con la depuración y la implementación .

Analice y compare estos patrones de flujo de trabajo con LangSmith . Siga la guía de inicio rápido para ver cómo fluyen los datos en cada paso. Le recomendamos configurar también LangSmith Engine , que supervisa los registros, detecta problemas y propone soluciones.

Configuración

LangChain Docs

1. Instalar dependencias:

```
pip install langchain_core langchain-anthropic langgraph
```

2. Inicializar el LLM:

```
import os
import getpass

from langchain_anthropic import ChatAnthropic

def _set_env(var: str):
    if not os.environ.get(var):
        os.environ[var] = getpass.getpass(f"{var}: ")

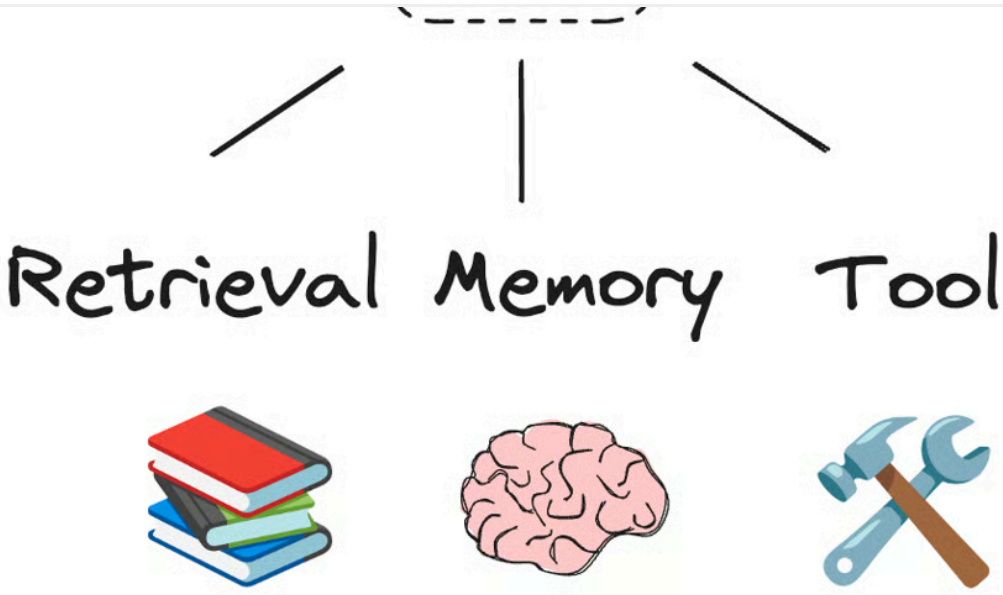

_set_env("ANTHROPIC_API_KEY")

llm = ChatAnthropic(model="claude-sonnet-4-6")
```

Maestrías en Derecho y aumentos

Los flujos de trabajo y los sistemas basados en agentes se fundamentan en LLM y las diversas ampliaciones que se les añaden. La llamada a herramientas , las salidas estructuradas y la memoria a corto plazo son algunas de las opciones para adaptar los LLM a sus necesidades.

LangChain Docs



LangChain Docs

```python
class SearchQuery(BaseModel):
    search_query: str | None = Field(
        default=None, description="Query that is optimized web search."
    )
    justification: str | None = Field(
        default=None, description="Why this query is relevant to the user's req
    )


# Augment the LLM with schema for structured output
structured_llm = llm.with_structured_output(SearchQuery)

# Invoke the augmented LLM
output = structured_llm.invoke("How does Calcium CT score relate to high choles
print(output)  # The model returns an instance of SearchQuery.

# Define a tool
def multiply(a: int, b: int) -> int:
    return a * b

# Augment the LLM with tools
llm_with_tools = llm.bind_tools([multiply])

# Invoke the LLM with input that triggers the tool call
msg = llm_with_tools.invoke("What is 2 times 3?")
print(msg.tool_calls)  # The model returns a request to call the tool.
```

encadenamiento de indicaciones

El encadenamiento de solicitudes se produce cuando cada llamada a LLM procesa la salida de la llamada anterior. Se suele utilizar para realizar tareas bien definidas que se pueden dividir en pasos más pequeños y verificables. Algunos ejemplos incluyen:

- Traducción de documentos a diferentes idiomas
- Verificar la coherencia del contenido generado

LangChain Docs



LangChain Docs

```
from IPython.display import Image, display


# Graph state
class State(TypedDict):
    topic: str
    joke: str
    improved_joke: str
    final_joke: str


# Nodes
def generate_joke(state: State):
    """First LLM call to generate initial joke"""

    msg = llm.invoke(f"Write a short joke about {state['topic']}")
    return {"joke": msg.content}


def check_punchline(state: State):
    """Gate function to check if the joke has a punchline"""

    # Simple check - does the joke contain "?" or "!"
    if "?" in state["joke"] or "!" in state["joke"]:
        return "Pass"
    return "Fail"


def improve_joke(state: State):
    """Second LLM call to improve the joke"""

    msg = llm.invoke(f"Make this joke funnier by adding wordplay: {state['joke
    return {"improved_joke": msg.content}


def polish_joke(state: State):
    """Third LLM call for final polish"""
    msg = llm.invoke(f"Add a surprising twist to this joke: {state['improved_j
    return {"final_joke": msg.content}
```

LangChain Docs

```python
# Add nodes
workflow.add_node("generate_joke", generate_joke)
workflow.add_node("improve_joke", improve_joke)
workflow.add_node("polish_joke", polish_joke)

# Add edges to connect nodes
workflow.add_edge(START, "generate_joke")
workflow.add_conditional_edges(
    "generate_joke", check_punchline, {"Fail": "improve_joke", "Pass": END}
)
workflow.add_edge("improve_joke", "polish_joke")
workflow.add_edge("polish_joke", END)

# Compile
chain = workflow.compile()

# Show workflow
display(Image(chain.get_graph().draw_mermaid_png()))

# Invoke
state = chain.invoke({"topic": "cats"})
print("Initial joke:")
print(state["joke"])
print("\n--- --- ---\n")
if "improved_joke" in state:
    print("Improved joke:")
    print(state["improved_joke"])
    print("\n--- --- ---\n")

    print("Final joke:")
    print(state["final_joke"])
else:
    print("Final joke:")
    print(state["joke"])
```

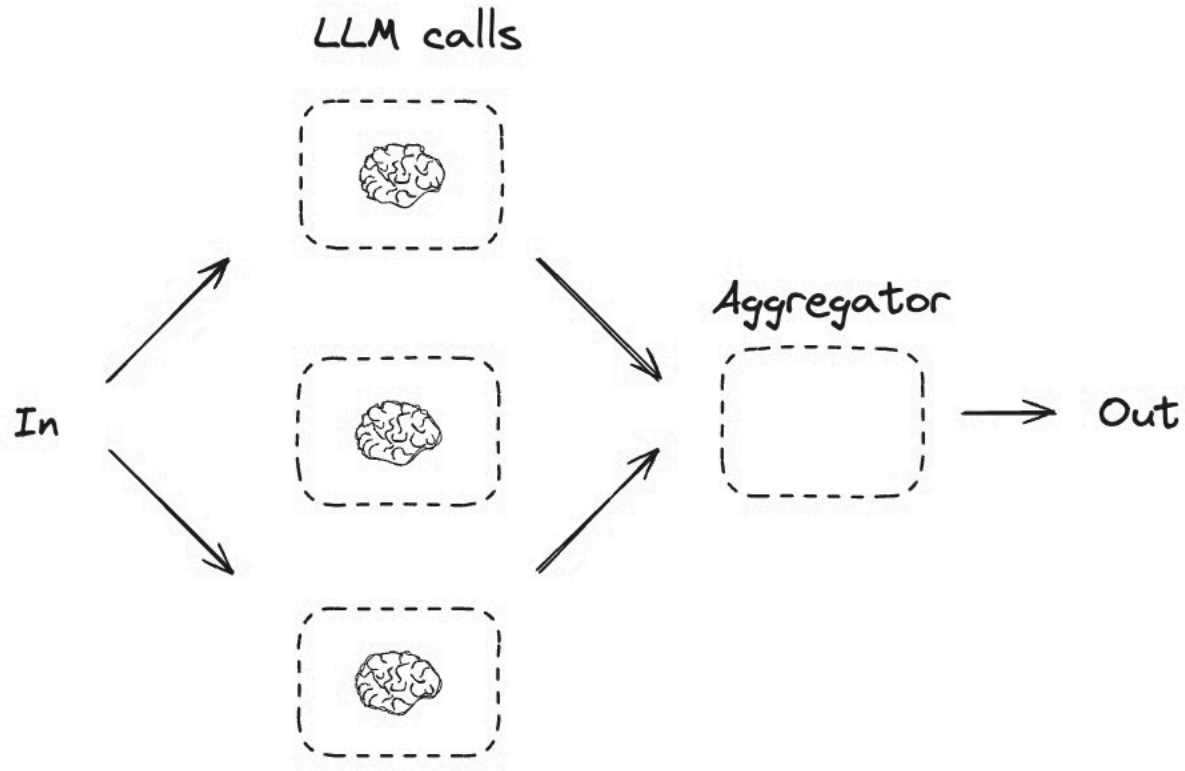
Paralelización

LangChain Docs

- Divide las subtareas y ejecútalas en paralelo, lo que aumenta la velocidad.
- Ejecuta las tareas varias veces para comprobar diferentes resultados, lo que aumenta la confianza.

Algunos ejemplos incluyen:

- Ejecutar una subtarea que procesa un documento en busca de palabras clave y una segunda subtarea para comprobar si hay errores de formato.
- Ejecutar varias veces una tarea que evalúa la precisión de un documento según diferentes criterios, como el número de citas, el número de fuentes utilizadas y la calidad de las fuentes.



LangChain Docs

```
    topic: str
    joke: str
    story: str
    poem: str
    combined_output: str


# Nodes
def call_llm_1(state: State):
    """First LLM call to generate initial joke"""

    msg = llm.invoke(f"Write a joke about {state['topic']}")
    return {"joke": msg.content}


def call_llm_2(state: State):
    """Second LLM call to generate story"""

    msg = llm.invoke(f"Write a story about {state['topic']}")
    return {"story": msg.content}


def call_llm_3(state: State):
    """Third LLM call to generate poem"""

    msg = llm.invoke(f"Write a poem about {state['topic']}")
    return {"poem": msg.content}


def aggregator(state: State):
    """Combine the joke, story and poem into a single output"""

    combined = f"Here's a story, joke, and poem about {state['topic']}!\n\n"
    combined += f"STORY:\n{state['story']}\n\n"
    combined += f"JOKE:\n{state['joke']}\n\n"
    combined += f"POEM:\n{state['poem']}"
    return {"combined_output": combined}
```

LangChain Docs

```
parallel_builder.add_node("call_llm_2", call_llm_2)
parallel_builder.add_node("call_llm_3", call_llm_3)
parallel_builder.add_node("aggregator", aggregator)

# Add edges to connect nodes
parallel_builder.add_edge(START, "call_llm_1")
parallel_builder.add_edge(START, "call_llm_2")
parallel_builder.add_edge(START, "call_llm_3")
parallel_builder.add_edge("call_llm_1", "aggregator")
parallel_builder.add_edge("call_llm_2", "aggregator")
parallel_builder.add_edge("call_llm_3", "aggregator")
parallel_builder.add_edge("aggregator", END)
parallel_workflow = parallel_builder.compile()

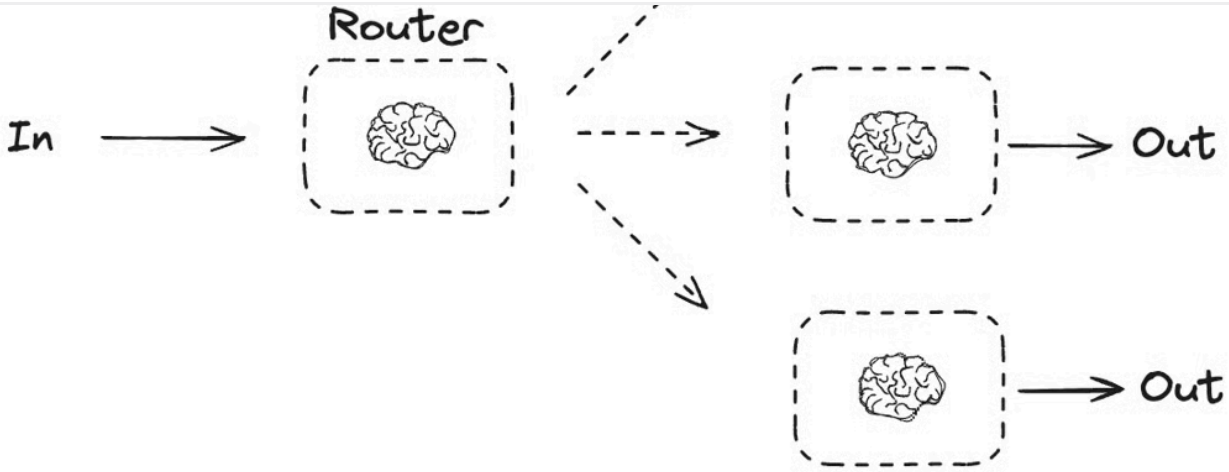
# Show workflow
display(Image(parallel_workflow.get_graph().draw_mermaid_png()))

# Invoke
state = parallel_workflow.invoke({"topic": "cats"})
print(state["combined_output"])
```

Enrutamiento

Los flujos de trabajo de enrutamiento procesan las entradas y las dirigen a tareas específicas según el contexto. Esto permite definir flujos especializados para tareas complejas. Por ejemplo, un flujo de trabajo diseñado para responder preguntas relacionadas con productos podría procesar primero el tipo de pregunta y luego enrutar la solicitud a procesos específicos para precios, reembolsos, devoluciones, etc.

LangChain Docs



LangChain Docs

```python
# Schema for structured output to use as routing logic
class Route(BaseModel):
    step: Literal["poem", "story", "joke"] = Field(
        None, description="The next step in the routing process"
    )


# Augment the LLM with schema for structured output
router = llm.with_structured_output(Route)


# State
class State(TypedDict):
    input: str
    decision: str
    output: str


# Nodes
def llm_call_1(state: State):
    """Write a story"""

    result = llm.invoke(state["input"])
    return {"output": result.content}


def llm_call_2(state: State):
    """Write a joke"""

    result = llm.invoke(state["input"])
    return {"output": result.content}


def llm_call_3(state: State):
    """Write a poem"""

    result = llm.invoke(state["input"])
```

LangChain Docs

```
    # Run the augmented LLM with structured output to serve as routing logic
    decision = router.invoke(
        [
            SystemMessage(
                content="Route the input to story, joke, or poem based on the
            ),
            HumanMessage(content=state["input"]),
        ]
    )

    return {"decision": decision.step}


# Conditional edge function to route to the appropriate node
def route_decision(state: State):
    # Return the node name you want to visit next
    if state["decision"] == "story":
        return "llm_call_1"
    elif state["decision"] == "joke":
        return "llm_call_2"
    elif state["decision"] == "poem":
        return "llm_call_3"


# Build workflow
router_builder = StateGraph(State)

# Add nodes
router_builder.add_node("llm_call_1", llm_call_1)
router_builder.add_node("llm_call_2", llm_call_2)
router_builder.add_node("llm_call_3", llm_call_3)
router_builder.add_node("llm_call_router", llm_call_router)

# Add edges to connect nodes
router_builder.add_edge(START, "llm_call_router")
router_builder.add_conditional_edges(
    "llm_call_router",
```

LangChain Docs

```
    },
)
router_builder.add_edge("llm_call_1", END)
router_builder.add_edge("llm_call_2", END)
router_builder.add_edge("llm_call_3", END)

# Compile workflow
router_workflow = router_builder.compile()

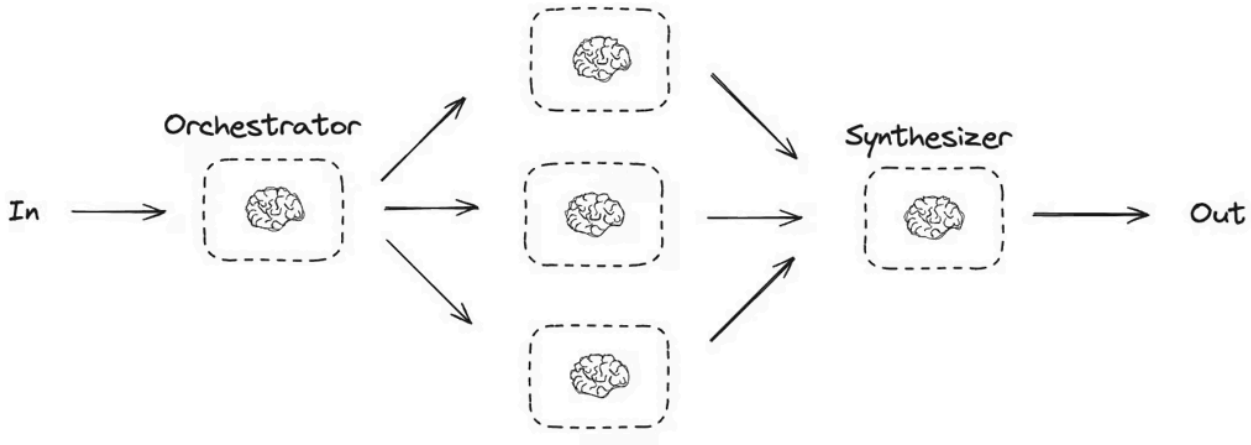
# Show the workflow
display(Image(router_workflow.get_graph().draw_mermaid_png()))

# Invoke
state = router_workflow.invoke({"input": "Write me a joke about cats"})
```

Orquestador-trabajador

En una configuración orquestador-trabajador, el orquestador:

- Descompone las tareas en subtareas.
- Delega subtareas a los trabajadores.
- Sintetiza los resultados de los trabajadores en un resultado final.



instrucciones de instalación de varias bibliotecas de Python en un número indeterminado de documentos podría utilizar este patrón.

API de gráficos API funcional

```python
from typing import Annotated, List
import operator


# Schema for structured output to use in planning
class Section(BaseModel):
    name: str = Field(
        description="Name for this section of the report.",
    )
    description: str = Field(
        description="Brief overview of the main topics and concepts to be cove
    )


class Sections(BaseModel):
    sections: List[Section] = Field(
        description="Sections of the report.",
    )


# Augment the LLM with schema for structured output
planner = llm.with_structured_output(Sections)
```

Creación de trabajadores en LangGraph

Los flujos de trabajo entre orquestadores y trabajadores son comunes, y LangGraph ofrece soporte integrado para ellos. La `Send` API permite crear nodos de trabajo dinámicamente y enviarles entradas específicas. Cada trabajador tiene su propio estado, y todas sus salidas se registran en una clave de estado compartida a la que tiene acceso el grafo del orquestador. Esto le da al orquestador acceso a todas las salidas de los

LangChain Docs

LangChain Docs

```python
class State(TypedDict):
    topic: str  # Report topic
    sections: list[Section]  # List of report sections
    completed_sections: Annotated[
        list, operator.add
    ]  # All workers write to this key in parallel
    final_report: str  # Final report


# Worker state
class WorkerState(TypedDict):
    section: Section
    completed_sections: Annotated[list, operator.add]


# Nodes
def orchestrator(state: State):
    """Orchestrator that generates a plan for the report"""

    # Generate queries
    report_sections = planner.invoke(
        [
            SystemMessage(content="Generate a plan for the report."),
            HumanMessage(content=f"Here is the report topic: {state['topic']}")
        ]
    )

    return {"sections": report_sections.sections}


def llm_call(state: WorkerState):
    """Worker writes a section of the report"""

    # Generate section
    section = llm.invoke(
        [
            SystemMessage(
                content="Write a report section following the provided name and
            ),
```

LangChain Docs

```
    # Write the updated section to completed sections
    return {"completed_sections": [section.content]}


def synthesizer(state: State):
    """Synthesize full report from sections"""

    # List of completed sections
    completed_sections = state["completed_sections"]

    # Format completed section to str to use as context for final sections
    completed_report_sections = "\n\n---\n\n".join(completed_sections)

    return {"final_report": completed_report_sections}


# Conditional edge function to create llm_call workers that each write a sectic
def assign_workers(state: State):
    """Assign a worker to each section in the plan"""

    # Kick off section writing in parallel via Send() API
    return [Send("llm_call", {"section": s}) for s in state["sections"]]


# Build workflow
orchestrator_worker_builder = StateGraph(State)

# Add the nodes
orchestrator_worker_builder.add_node("orchestrator", orchestrator)
orchestrator_worker_builder.add_node("llm_call", llm_call)
orchestrator_worker_builder.add_node("synthesizer", synthesizer)

# Add edges to connect nodes
orchestrator_worker_builder.add_edge(START, "orchestrator")
orchestrator_worker_builder.add_conditional_edges(
    "orchestrator", assign_workers, ["llm_call"]
)
```

LangChain Docs

```
# Show the workflow
display(Image(orchestrator_worker.get_graph().draw_mermaid_png()))

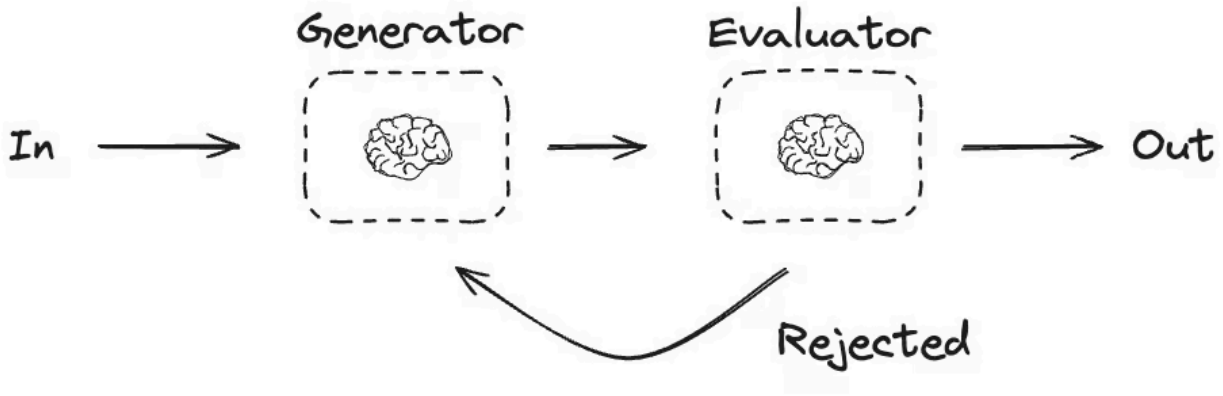
# Invoke
state = orchestrator_worker.invoke({"topic": "Create a report on LLM scaling la

from IPython.display import Markdown
Markdown(state["final_report"])
```

Evaluador-optimizador

En los flujos de trabajo de evaluador-optimizador, una llamada a LLM genera una respuesta y la otra la evalúa. Si el evaluador o un operador humano determina que la respuesta necesita ajustes, se proporciona retroalimentación y se vuelve a generar. Este ciclo continúa hasta que se obtiene una respuesta aceptable.

Los flujos de trabajo de evaluación-optimización se utilizan habitualmente cuando existen criterios de éxito específicos para una tarea, pero se requiere iteración para cumplirlos. Por ejemplo, no siempre se obtiene una traducción perfecta entre dos idiomas. Puede que se necesiten varias iteraciones para generar una traducción con el mismo significado en ambos idiomas.



LangChain Docs

```
    joke: str
    topic: str
    feedback: str
    funny_or_not: str


# Schema for structured output to use in evaluation
class Feedback(BaseModel):
    grade: Literal["funny", "not funny"] = Field(
        description="Decide if the joke is funny or not.",
    )
    feedback: str = Field(
        description="If the joke is not funny, provide feedback on how to impr
    )


# Augment the LLM with schema for structured output
evaluator = llm.with_structured_output(Feedback)


# Nodes
def llm_call_generator(state: State):
    """LLM generates a joke"""

    if state.get("feedback"):
        msg = llm.invoke(
            f"Write a joke about {state['topic']} but take into account the fe
        )
    else:
        msg = llm.invoke(f"Write a joke about {state['topic']}")
    return {"joke": msg.content}


def llm_call_evaluator(state: State):
    """LLM evaluates the joke"""

    grade = evaluator.invoke(f"Grade the joke {state['joke']}")
    return {"funny_or_not": grade.grade, "feedback": grade.feedback}
```

LangChain Docs

```
    if state["funny_or_not"] == "funny":
        return "Accepted"
    elif state["funny_or_not"] == "not funny":
        return "Rejected + Feedback"


# Build workflow
optimizer_builder = StateGraph(State)

# Add the nodes
optimizer_builder.add_node("llm_call_generator", llm_call_generator)
optimizer_builder.add_node("llm_call_evaluator", llm_call_evaluator)

# Add edges to connect nodes
optimizer_builder.add_edge(START, "llm_call_generator")
optimizer_builder.add_edge("llm_call_generator", "llm_call_evaluator")
optimizer_builder.add_conditional_edges(
    "llm_call_evaluator",
    route_joke,
    {  # Name returned by route_joke : Name of next node to visit
        "Accepted": END,
        "Rejected + Feedback": "llm_call_generator",
    },
)

# Compile the workflow
optimizer_workflow = optimizer_builder.compile()

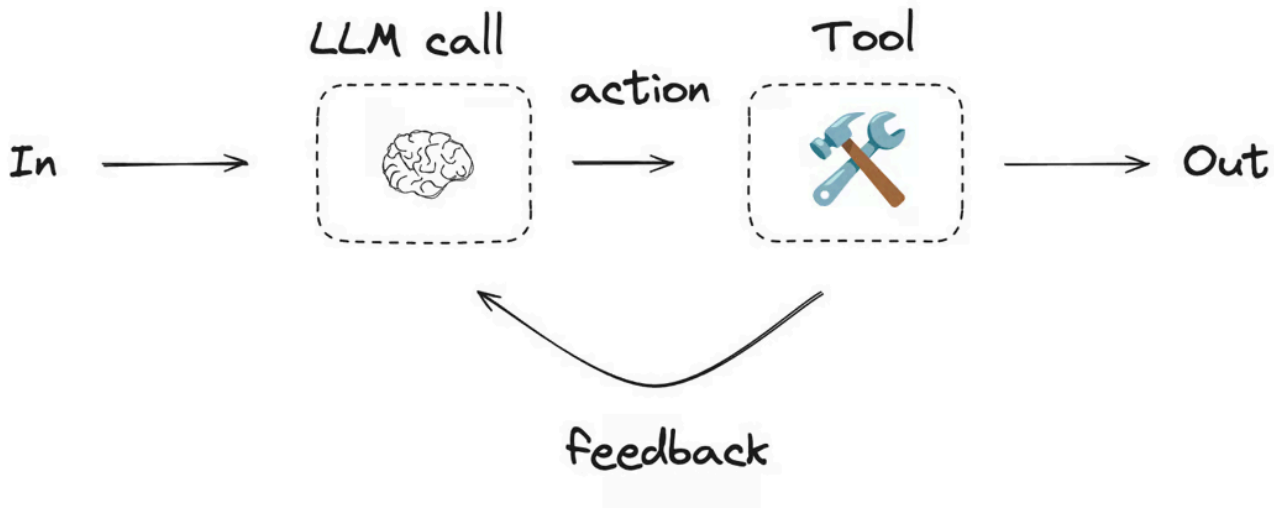
# Show the workflow
display(Image(optimizer_workflow.get_graph().draw_mermaid_png()))

# Invoke
state = optimizer_workflow.invoke({"topic": "Cats"})
print(state["joke"])
```

Agentes

LangChain Docs

sobre las herramientas que utilizan y cómo resolver problemas. Aun así, es posible definir el conjunto de herramientas disponibles y las directrices sobre el comportamiento de los agentes.



Para empezar a usar agentes, consulta la guía de inicio rápido o lee más sobre cómo funcionan en LangChain.

LangChain Docs

```python
# Define tools
@tool
def multiply(a: int, b: int) -> int:
    """Multiply `a` and `b`.

    Args:
        a: First int
        b: Second int
    """
    return a * b


@tool
def add(a: int, b: int) -> int:
    """Adds `a` and `b`.

    Args:
        a: First int
        b: Second int
    """
    return a + b


@tool
def divide(a: int, b: int) -> float:
    """Divide `a` and `b`.

    Args:
        a: First int
        b: Second int
    """
    return a / b


# Augment the LLM with tools
tools = [add, multiply, divide]
```

LangChain Docs

LangChain Docs

```python
# Nodes
def llm_call(state: MessagesState):
    """LLM decides whether to call a tool or not"""

    return {
        "messages": [
            llm_with_tools.invoke(
                [
                    SystemMessage(
                        content="You are a helpful assistant tasked with perfo
                    )
                ]
                + state["messages"]
            )
        ]
    }


def tool_node(state: MessagesState):
    """Performs the tool call"""

    result = []
    for tool_call in state["messages"][-1].tool_calls:
        tool = tools_by_name[tool_call["name"]]
        observation = tool.invoke(tool_call["args"])
        result.append(ToolMessage(content=observation, tool_call_id=tool_call[
    return {"messages": result}


# Conditional edge function to route to the tool node or end based upon whethe
def should_continue(state: MessagesState) -> Literal["tool_node", END]:
    """Decide if we should continue the loop or stop based upon whether the LL

    messages = state["messages"]
    last_message = messages[-1]

    # If the LLM makes a tool call, then perform an action
```

LangChain Docs

```
# Build workflow
agent_builder = StateGraph(MessagesState)

# Add nodes
agent_builder.add_node("llm_call", llm_call)
agent_builder.add_node("tool_node", tool_node)

# Add edges to connect nodes
agent_builder.add_edge(START, "llm_call")
agent_builder.add_conditional_edges(
    "llm_call",
    should_continue,
    ["tool_node", END]
)
agent_builder.add_edge("tool_node", "llm_call")

# Compile the agent
agent = agent_builder.compile()

# Show the agent
display(Image(agent.get_graph(xray=True).draw_mermaid_png()))

# Invoke
messages = [HumanMessage(content="Add 3 and 4.")]
messages = agent.invoke({"messages": messages})
for m in messages["messages"]:
    m.pretty_print()
```

ToolNode

`ToolNode` Es un nodo preconfigurado que ejecuta herramientas en flujos de trabajo de LangGraph. Gestiona automáticamente la ejecución paralela de herramientas, el manejo de errores y la inyección de estado.

LangChain Docs

```python
from langchain.tools import tool
from langgraph.prebuilt import ToolNode
from langgraph.graph import MessagesState, StateGraph

@tool
def search(query: str) -> str:
    """Search for information."""
    return f"Results for: {query}"

@tool
def calculator(expression: str) -> str:
    """Evaluate a math expression."""
    return str(eval(expression))

builder = StateGraph(MessagesState)
builder.add_node("tools", ToolNode([search, calculator]))
# ... add other nodes and edges
graph = builder.compile()
```

Acceda al estado y al contexto del gráfico desde las herramientas

Las herramientas ejecutadas `ToolNode` reciben los argumentos generados por el modelo como su primer argumento. Para leer datos del lado del gráfico que no fueron generados por el modelo, utilice una de estas opciones:

- En Python, lee el estado y el contexto de ejecución a partir del `ToolRuntime` argumento inyectado.
- En JavaScript, lea el estado y el contexto de ejecución desde el segundo argumento de la herramienta, escrito como `ToolRuntime` .

LangChain Docs

estado completo, mientras que pasar solo `{"messages": state["messages"]}` o `{ messages: state.messages }` solo expone `messages` .

LangChain Docs

```python
from langgraph.graph import MessagesState, START, StateGraph
from langgraph.prebuilt import ToolNode


class State(MessagesState):
    user_id: str


@dataclass
class Context:
    organization_id: str


@tool
def get_user_info(runtime: ToolRuntime[Context, State]) -> str:
    """Look up user information."""
    # Read the current graph state passed to the ToolNode.
    user_id = runtime.state["user_id"]

    # Read explicit per-run values that are not part of graph state.
    organization_id = runtime.context.organization_id

    return f"User {user_id} in organization {organization_id}"


builder = StateGraph(State, context_schema=Context)
builder.add_node("tools", ToolNode([get_user_info]))
builder.add_edge(START, "tools")
graph = builder.compile()

result = graph.invoke(
    {
        "messages": [
            AIMessage(
                content="",
                tool_calls=[
                    {
                        "name": "get_user_info",
                        "args": {},
```

LangChain Docs

```
        ],
        "user_id": "user_123",
    },
    context=Context(organization_id="org_456"),
)
```

Ver ejemplo de traza
Abra una ejecución pública de LangSmith para este ejemplo.

Conecta estos documentos con el agente que prefieras a través de MCP para obtener respuestas en tiempo real.

Edita esta página en GitHub o crea una incidencia .

¿Te ha resultado útil esta página? Sí No

LangChain Docs